

LabReSiD26 - Hands-On 4

Davide Ferrara - 518629

1 Esercizio 1: Stampa leggibile del protocollo

Il campo `ip->protocol` contiene il numero di protocollo del livello di trasporto incapsulato nel pacchetto IP. Nel codice di partenza veniva stampato come numero intero con `%d`. La funzione `proto_name()` converte il numero in una stringa leggibile tramite uno switch case che matcha il numero del protocollo e ritorna la stringa corrispondente:

```
1 const char *proto_name(uint8_t protocol) {  
2     switch (protocol) {  
3         case 1:  
4             return "ICMP";  
5         case 6:  
6             return "TCP";  
7         case 17:  
8             return "UDP";  
9         default:  
10            return "sconosciuto";  
11     }  
12 }
```

Listing 1: `proto_name()` – implementazione

La `printf` è stata modificata per usare `proto_name()` al posto di `%d`, mantenendo il numero tra parentesi per completezza:

```
1 printf("  PROTO: %s (%d)\n", proto_name(ip->protocol),  
        ip->protocol);
```

Listing 2: Uso nel loop di stampa

```
1  PROTO: ICMP (1)
```

Listing 3: Output

2 Esercizio 2: Decodifica completa dell'header IP

Ho esteso il programma per stampare tutti i campi significativi dell'header IP. È importante segnalare che se `ip->ihl < 5` il pacchetto viene scartato con `continue` nel ciclo, in quanto un valore inferiore a 5 indica un header malformato (la lunghezza minima valida è $5 \times 4 = 20$ byte).

Il campo `frag_off` è un `uint16_t` che comprime tre informazioni:

| bit 15 (riservato) | bit 14 (DF) | bit 13 (MF) | bit 12--0
(offset) |

- **DF** (Don't Fragment, bit 14): il pacchetto non deve essere frammentato.
- **MF** (More Fragments, bit 13): esistono altri frammenti successivi.
- **Fragment Offset** (bit 12-0, 13 bit): posizione del frammento in unità da 8 byte.

Si estraggono con operazioni bitwise dopo aver convertito `frag_off` in host byte order con `ntohs()`:

```
1 uint16_t fo = ntohs(ip->frag_off);
2 int flag_df = (fo >> 14) & 1; /* sposta bit 14 in pos.
   0, prendi solo quello */
3 int flag_mf = (fo >> 13) & 1; /* stesso per bit 13 */
4 int offset = fo & 0x1FFF; /* maschera i 13 bit bassi
   */
```

Listing 4: Estrazione flag e offset da `frag_off`

```
1 if (ip->ihl < 5) {
2     printf("[SNIFFER.c] Pacchetto malformato\n");
3     continue;
4 }
5
6 uint16_t tot_len = ntohs(ip->tot_len);
7 int len = ip->ihl * 4;
8 uint16_t fo = ntohs(ip->frag_off);
9 int flag_df = (fo >> 14) & 1; /* Don't Fragment */
10 int flag_mf = (fo >> 13) & 1; /* More Fragments */
11 int offset = fo & 0x1FFF; /* Fragment Offset (in
   unita' da 8 byte) */
```

```

12
13 printf("  VERSION: %d\n",    ip->version);
14 printf("  IHL: %d byte\n",    len);
15 printf("  LUNGHEZZA: %d byte\n", tot_len);
16 printf("  TTL: %d\n",        ip->ttl);
17 printf("  PROTO: %s (%d)\n", proto_name(ip->protocol),
        ip->protocol);
18 printf("  SRC: %s\n",        inet_ntoa(s));
19 printf("  DST: %s\n",        inet_ntoa(d));
20 printf("  FLAGS: %s%s\n",    flag_df ? "DF " : "",
        flag_mf ? "MF" : "");
21 printf("  FRAG OFF: %d\n\n", fo);

```

Listing 5: Decodifica completa header IP

```

1 sudo ./sniffer
2 In ascolto (solo ICMP)... premi Ctrl+C per uscire
3
4 Pacchetto #1  (84 byte)
5   VERSION: 4
6   IHL: 20 byte
7   LUNGHEZZA: 84 byte
8   TTL: 116
9   PROTO: ICMP (1)
10  SRC: 8.8.8.8
11  DST: 192.168.8.230
12  FLAGS:
13  FRAG OFF: 0

```

Listing 6: Output – ping -c 3 8.8.8.8

FLAGS è vuoto e FRAG OFF: 0 perché il pacchetto non è frammentato e il flag DF non è settato nella risposta di 8.8.8.8.

3 Challenge: Naviga nel payload – decodifica l'header ICMP

La funzione `ip_payload()` calcola il puntatore all'inizio del payload IP tenendo conto delle eventuali opzioni (`ihl > 5`) ed esegue due controlli di sicurezza prima di restituire il puntatore:

- `ip->ihl < 5`: header malformato, lunghezza impossibile.

- $n < hlen$: pacchetto troncato — `recvfrom()` ha consegnato meno byte di quanti l'header dichiara. Su IP non esiste ritrasmissione, il pacchetto va scartato.

```

1 unsigned char *ip_payload(const unsigned char *buf,
    ssize_t n,
2                               size_t *payload_len) {
3     struct iphdr *ip = (struct iphdr *)buf;
4     if (ip->ihl < 5)
5         return NULL; /* malformato */
6     int hlen = ip->ihl * 4;
7     if (n < hlen) /* se n < hlen e accedo a buf+hlen vado
8         in segfault */
9         return NULL; /* troncato: in IP non esiste
10            ritrasmissione, lo scarto */
11     *payload_len = n - hlen;
12     return (unsigned char *)buf + hlen;
13 }

```

Listing 7: `ip_payload()` – implementazione

```

1 size_t payload_len;
2 unsigned char *payload = ip_payload(buf, n, &payload_len
3     );
4 if (payload == NULL)
5     continue;
6 printf("  PAYLOAD:\n");
7 if (ip->protocol == 1) {
8     struct icmphdr *icmp = (struct icmphdr *)payload;
9     printf("  TYPE:      %d\n", icmp->type);
10    printf("  CODE:      %d\n", icmp->code);
11    printf("  CHECKSUM: %d\n", ntohs(icmp->checksum));
12    printf("  ID:        %d\n", ntohs(icmp->un.echo.id));
13    printf("  SEQ:      %d\n\n", ntohs(icmp->un.echo.
14        sequence));
15 }

```

Listing 8: Uso nel loop – decodifica header ICMP

```
1  PAYLOAD :  
2  TYPE : 0  
3  ID : 1423  
4  SEQ : 256
```

Listing 9: Output – ICMP Echo Reply da 8.8.8.8